PROJECT G · REMOTE-WORK COMPANION

Doing the Project From Home & In the Lab

What to build on your laptop, what needs the Orin, and how the work travels between them

The core idea of this guide

Most of Project G is **logic and learning**, and that can be built on your laptop. Only a few things truly need the Orin: building TensorRT **engines**, the **camera**, and measuring **real-time speed**. So the working pattern is: **develop and test the logic at home, then carry it to the Orin for the device-specific and speed-specific steps.** This companion splits every day into exactly that.

If your laptop has no NVIDIA GPU

That's completely fine. Everything in the **at-home** sections still works — it just runs on your laptop's CPU, which is slower but produces the same results. You only need a GPU for **speed**, and the Orin provides that. Don't let “my laptop isn't powerful” stop you; for learning and building logic, a plain laptop is enough.

Contents

1. Set up your laptop (do this once)

Before any home work, your laptop needs Python and a clean place to install the project's tools. This takes about fifteen minutes and you only do it once. Treating you as a total beginner, here's every step.

Step 1 — Check you have Python

Open your laptop's terminal (on Windows, open “PowerShell” or “Terminal”; on Mac, open “Terminal”; on Linux you know where it is). Type `python3 --version` and press Enter. If it prints a version like `Python 3.11.x`, you're set. If it says “not found,” try `python --version`. If neither works, install Python from python.org first.

Step 2 — Make a project folder and a “virtual environment”

Concept: what is a virtual environment?

A **virtual environment** (“venv”) is a private sandbox folder for one project's Python packages, so they don't clash with the rest of your system. You create it once, then “activate” it whenever you work on the project. When it's active, anything you `pip install` goes only into that sandbox.

Create and enter the project + venv

```
mkdir project_g          # make a folder for the project
cd project_g             # go into it
python3 -m venv venv     # create the sandbox (one time)
```

Now **activate** the sandbox. The command differs by operating system:

Activate the venv (every time you open a new terminal)

```
# Mac / Linux:
source venv/bin/activate

# Windows (PowerShell):
venv\Scripts\Activate.ps1
```

When it works, you'll see `(venv)` appear at the start of your prompt. That's how you know the sandbox is active. You must re-activate it each time you open a new terminal window.

Step 3 — Install YOLO and test it

Install + first test (with the venv active)

```
pip install ultralytics
yolo predict model=yolo11n.pt source='https://ultralytics.com/images/bus.jpg'
```

The first install pulls in PyTorch and a lot of other tools — let it finish. The predict command downloads a small model and runs it on a sample image, saving an annotated copy (it prints the folder). If you see boxes drawn on a bus and some people, your laptop is ready to develop from.

2. How your work travels to the Orin

You build things at home; they need to reach the Orin in the lab. The tool for this is **Git** — the same version control the brief asks you to use daily. Understanding this is what makes remote work actually work.

Concept: Git, a repo, and the remote

Git tracks the history of a folder of code. That tracked folder is a **repository** (“repo”). When the repo also lives online (on GitHub or the lab's Git server — the “remote”), both your laptop and the Orin can reach the same code.

The everyday loop: you write code on your laptop, save it into the repo's history, and push it online. Then on the Orin you pull it down and run it. That push-then-pull is literally how your home work “arrives” at the device.

On your laptop, after doing some work

```
git add .                                  # stage your changes
git commit -m "add YOLO->prompt glue"      # save them with a message
git push                                   # send them online
```

On the Orin, to get your latest work

```
git pull                 # download the latest code
python3 your_script.py   # run it on the device
```

One important rule about Git

Git is for **code**, not big files. Do **not** put TensorRT engines, datasets, or model checkpoints in Git — they're huge and don't belong there. Those move a different way (a shared drive or the lab's storage; ask your lead). Engines also can't be reused across machines anyway, so you rebuild them on the Orin rather than carrying them over.

3. Which days are home, lab, or mixed

A planning overview. Many days are mostly home or mixed — so being away from the lab doesn't stop you making real progress.

Day	Where	What the day is about
1	Done	Boot the Orin; understand the mission
2	Home + Lab	Run YOLO at home; build its engine in the lab
3	Home + Lab	Learn segmentation at home; build NanoSAM engines in the lab
4	Mostly home	Design the YOLO→prompt glue
5	Mostly lab	Benchmark on the device (prep the script at home)
6	Home + Lab	Build the single-frame pipeline; swap in NanoSAM in the lab
7	On-site	Capture footage (plan + organise at home)
8	Mostly home	Label data and build the evaluation set
9	Mostly home	Find failure modes on real frames
10	Home + Lab	Demo on the device; write-up and re-plan at home
11	Mostly lab	FP16 engines on the Orin (understand precision at home)
12	Mostly lab	INT8 calibration on the Orin (build the set at home)
13	Home + Lab	Resolution accuracy at home; speed in the lab
14	Mostly home	Write + test the ROS 2 node with a fake camera
15	Mostly lab	Live integration in the full stack

Day	Where	What the day is about
16	Mostly home	Write + verify the 2D→3D projection math
17	Home + Lab	Conflict logic at home; persistence in the lab
18	Mostly home	Evaluation metrics at home; speed figure in the lab
19	Home	Documentation and repo tidy-up
20	Mostly home	Build the talk; record the demo video in the lab

4. The 20 days — home and lab, side by side

Each day gives the **goal**, **the gist** (what you're really doing, in plain terms), the **readings**, what to do **at home on your laptop**, what waits for **the Orin in the lab**, and what to **carry to the lab**. Use Git (Section 2) to move the carry-over items across.

WEEK 1
Foundations & first models

DAY 1 — Orientation & Environment Setup

Boot the Orin; understand the mission • DONE

Goal. Get the box running and your bearings — you've done this.

THE GIST — what you're really doing

You've already done the lab half: booting the Orin, running `nvidia-smi`, and finding the PyTorch/CUDA situation. The part that was always home-friendly was the **reading** — understanding the shift from "train my own model" to "prompt a pre-trained one." The most useful thing you can still do at home now is set up your laptop (next section) so every following day's home work is unblocked.

READINGS

- Foundation models overview — re-skim if the model names still feel fuzzy.

AT HOME — on your laptop

- Set up your laptop once, following "Set up your laptop" below — this is the real Day-1-at-home task.
- Re-read the foundation-models overview so the YOLO + NanoSAM idea is solid.

IN THE LAB — on the Orin

- (Done) environment checks; `nvidia-smi` shows the GPU.
- Make sure your day-one questions to the lead are answered, especially container-vs-native and the network to use.

CARRY TO THE LAB A working laptop environment, ready to develop from home.

DAY 2 — YOLO Running + TensorRT Export

Get the detector working and understood • HOME + LAB

Goal. Run YOLO on images and learn exactly what it outputs.

THE GIST — what you're really doing

YOLO is the detector — the piece that finds and names objects. The wonderful thing for you right now: **YOLO runs on any computer, including your laptop**. So at home you install it, run it on images, and learn precisely how to call it and what it hands back (names, boxes, confidence scores). The **only** thing that must wait for the Orin is compiling YOLO into a fast TensorRT "engine" and measuring its speed — because an engine is built for one specific GPU and cannot be copied between machines. So: learn the behaviour at home, build the fast version in the lab.

READINGS

- YOLO on NVIDIA Jetson — Ultralytics — the install part differs on a laptop, but skim it.

- YOLO → TensorRT export — read now, run in the lab.

AT HOME — on your laptop

- In your laptop's virtual environment, install Ultralytics (command below).
- Run YOLO on a sample image and open the annotated result it saves.
- Run it on a few of your own photos to see it detect everyday objects — build intuition.
- In Python, load a model and print its outputs (snippet below). Learn the shapes of `.xyxy`, `.cls`, `.conf` — this is the knowledge you need to join YOLO to NanoSAM later.

Install + first detection (laptop terminal, venv active)

```
pip install ultralytics
yolo predict model=yolo11n.pt source='https://ultralytics.com/images/bus.jpg'
```

Inspect YOLO's output (Python)

```
from ultralytics import YOLO
model = YOLO('yolo11n.pt')
r = model('https://ultralytics.com/images/bus.jpg')
print(r[0].boxes.xyxy)   # box corners (x1,y1,x2,y2)
print(r[0].boxes.cls)    # class id per box
print(r[0].boxes.conf)   # confidence per box
```

IN THE LAB — on the Orin

- Build the TensorRT engine **on the Orin**: `yolo export model=yolo11n.pt format=engine half=True`.
- Run inference from the `.engine` and record FPS before vs after.

> **CARRY TO THE LAB** A solid grasp of YOLO's API and a small committed script that detects and prints boxes — the engine build in the lab is then just one command.

DAY 3

NanoSAM Running

Learn promptable segmentation; build engines in lab • HOME + LAB

> **Goal.** Understand how a box becomes a mask, and get masks out.

THE GIST — what you're really doing

NanoSAM turns a box into a precise mask. NanoSAM **itself** is Jetson-only — its fast pieces are TensorRT engines you build on the Orin — so the real thing waits for the lab. But the **idea** it's based on, "promptable segmentation," you can learn at home using its bigger cousins **SAM** or **MobileSAM**, which are ordinary PyTorch and run on a laptop (slowly on CPU, but fine for learning). So at home you experiment with "give a point or box, get a mask"; in the lab you build NanoSAM's engines and run the fast version. The predictor APIs are similar enough that what you learn carries over.

READINGS

- NanoSAM README — read the "build the engine" and predictor sections fully.
- Knowledge distillation intro — why NanoSAM is small and fast.

AT HOME — on your laptop

- Read the NanoSAM README end to end so you know the predictor flow: `set_image()` then `predict(points, labels)`.
- Learn the point-label convention: `1`=foreground, `0`=background, `2`/`3`=box corners.
- Optional, and **ask your lead first**: install MobileSAM or SAM on your laptop and run their point/box demo on an image, to feel how promptable segmentation behaves. SAM is a large download — only worth it if your laptop can handle it.

IN THE LAB — on the Orin

- Build NanoSAM's two engines (image encoder + mask decoder) with `trtexec`, following the README's exact flags.
- Run the predictor on a test image with a point prompt; save the mask; note latency.

CARRY TO THE LAB Confident understanding of the predictor API and label convention, so the lab build is pure mechanics.

DAY 4 — Understand the Combined Pipeline

Design the glue — almost all home work • MOSTLY HOME

Goal. Design and test the code that turns YOLO boxes into segmentation prompts.

THE GIST — what you're really doing

This is the single most home-friendly day. You're working out how YOLO's **boxes** become NanoSAM's **prompts** — pure logic, no special hardware. You can write and test the whole "glue" on your laptop with YOLO (which runs at home) and, if you set it up, SAM/MobileSAM standing in for NanoSAM. The conversion code you write at home will work unchanged on the Orin once NanoSAM is swapped in.

READINGS

- YOLO + SAM combined framework (paper) — focus on the detector-then-segmenter diagram.

AT HOME — on your laptop

- Sketch the data flow on paper: image → YOLO → (name, box) list → for each box → segmenter → mask.
- Write a conversion function that turns a YOLO box (`xyxy`) into the prompt format the segmenter expects (corner points + labels 2/3).
- Test it by printing inputs and outputs. If you have SAM at home, actually feed a converted box in and confirm you get a sensible mask.

IN THE LAB — on the Orin

- Swap your laptop's SAM stand-in for NanoSAM's predictor and confirm the same glue produces masks on the Orin.

CARRY TO THE LAB A tested conversion function (committed to Git) that turns a YOLO box into a segmentation prompt.

DAY 5 — Baseline Benchmarking + Week Review

Measure speed honestly (mostly on the Orin) • MOSTLY LAB

Goal. Get honest baseline speed numbers on the device.

THE GIST — what you're really doing

Benchmarking only means something on the **real device**, because speed depends on the Orin's specific GPU and how hot it gets. At home you can't get meaningful numbers — but you **can** learn what the numbers mean and write the timing-and-logging script so it's ready to run the instant you're at the Orin.

READINGS

- Jetson Orin overview — power modes and throttling.
- Benchmarking on Jetson

AT HOME — on your laptop

- Learn latency vs throughput vs real-time (the handbook's glossary covers this).
- Write a small timing harness: run a model N times, discard the warm-up frames, print average milliseconds and FPS. Test its logic on your laptop with YOLO.

IN THE LAB — on the Orin

- Enable max performance: `sudo nvpmodel -m 0` then `sudo jetson_clocks`.
- Run `tegrastats`; time each model over ~100 frames; record the table. Write the Week 1 update.

> **CARRY TO THE LAB** A ready-to-run benchmarking script, so lab time is spent collecting numbers, not writing code.

WEEK 2

Build the pipeline + gather data

DAY 6

Connect YOLO → NanoSAM (single frame)

Full detect-then-segment on one image • HOME + LAB

Goal. Get the whole pipeline correct on a single image.

THE GIST — what you're really doing

You build the actual pipeline script. At home you can get it **fully correct** using YOLO plus the SAM stand-in: load an image, detect, convert the boxes, segment each one, overlay the masks. In the lab you swap SAM for NanoSAM. Because the structure is identical, almost all the debugging happens comfortably at home, not under pressure on the device.

READINGS

- Re-read both predictor APIs (focus on input/output shapes).

AT HOME — on your laptop

- Write the pipeline: image → YOLO → for each box → segmenter → mask → overlay all masks, colour-coded, with labels.
- Handle the “no detections” case so it never crashes.
- Get the overlay visualization looking right; commit the script.

IN THE LAB — on the Orin

- Replace the segmenter with NanoSAM; run on the Orin; confirm masks line up with objects.

CARRY TO THE LAB The working pipeline script (Git) that needs only the segmenter swapped for NanoSAM.

DAY 7

Data Collection Planning + Capture

Capture real footage (on-site) • MOSTLY LAB

Goal. Gather varied real footage with the device.

THE GIST — what you're really doing

This needs the SCANNER-01, so the capture itself is an on-site task. At home you do the **preparation**: decide which conditions to capture, design a clear folder and naming scheme, and write a small helper script to organise recordings as they come in. Good preparation makes the on-site capture fast and the data usable.

READINGS

- The data-collection notes in the Project G brief — variety of conditions beats raw count.

AT HOME — on your laptop

- List the conditions that matter: times of day, weather, distances, scene types.
- Design a naming scheme, e.g. `2026-06-23_0830_overcast_field-edge`, and a folder structure.
- Write a small script to rename/organise recordings into that structure.

IN THE LAB — on the Orin

- Capture diverse footage with the SCANNER-01 across your planned conditions; organise it with your scheme.

CARRY TO THE LAB A capture plan and an organising script, ready to apply on-site.

DAY 8 — Evaluation Set + Labelling

Build ground truth (very home-friendly) • MOSTLY HOME

Goal. Create the labelled test set you'll score the pipeline against.

THE GIST — what you're really doing

Labelling and building the evaluation set are very home-friendly: **CVAT runs on your laptop**, so you can label images, define your class set, and create train/val/test splits at home. The only thing you need is images to label — from Day 7's capture, or sample images in the meantime. Remember the rule: split by whole recording **session**, never by individual frame, or your scores will lie.

READINGS

- CVAT getting started
- Why to split by session, not frame (data leakage).

AT HOME — on your laptop

- Install and run CVAT locally (Docker or the hosted version).
- Define your class set / text prompts — start with 5–8, and write a one-page labelling guideline.
- Label a batch of images; create the train/val/test split by session.

IN THE LAB — on the Orin

- Only if the footage lives on lab storage: copy the frames you need. Otherwise nothing — this day is yours to do remotely.

CARRY TO THE LAB A labelled evaluation set and a documented class/prompt list (in Git or shared storage).

DAY 9 — Pipeline Robustness on Real Frames

Find failure modes (home-doable) • MOSTLY HOME

Goal. Discover where the pipeline breaks on real footage.

THE GIST — what you're really doing

Once you have frames, running your pipeline across them and cataloguing the failures is laptop work — using the SAM stand-in. You're hunting for where it misses objects, makes bad masks, or chokes. Finding these at home, calmly, is far better than discovering them live in the lab.

READINGS

- Skim NanoSAM's notes on multiple prompts / batching (feeding several boxes per frame).

AT HOME — on your laptop

- Run your Day 6 pipeline over a whole folder of frames; save the annotated outputs and scroll through them.
- Catalogue failure modes: missed objects, wrong masks, tiny objects, overlaps.
- Fix the crashes first (empty detections, odd image sizes); note quality issues for later.

IN THE LAB — on the Orin

- Quick re-check on the Orin with NanoSAM that the same frames behave similarly.

CARRY TO THE LAB A written failure-mode list and crash fixes (Git).

DAY — Mid-Project Checkpoint

10 *Demo, feedback, re-plan* • HOME + LAB

Goal. Show what works and re-plan the back half honestly.

THE GIST — what you're really doing

Halfway. The live demo is best on the Orin, but the **preparation and reflection** are home work: writing your Week 2 update, preparing what you'll show, and re-planning Weeks 3–4 based on where you actually are.

READINGS

- Your own Week 1–2 notes and the failure-mode list.

AT HOME — on your laptop

- Write the Week 2 status update; prepare your demo talking points.
- Re-plan Weeks 3–4 realistically — if behind, decide now what to cut.

IN THE LAB — on the Orin

- Run the full pipeline on a short live walk and demo it to your lead, failure list included.

CARRY TO THE LAB A written update and a revised plan to review with your lead.

WEEK 3

Optimise + make it deployable

DAY 11 — Precision: FP16 Everywhere

First big speedup (on the Orin) • MOSTLY LAB

Goal. Make both models run in FP16 and find the bottleneck.

THE GIST — what you're really doing

Precision and engine work happen on the Orin, because engines are device-specific. At home you **understand** what FP16 means and prepare the exact export commands so the lab session is fast.

READINGS

- Quantization overview — PyTorch — FP32 vs FP16 vs INT8.
- TensorRT intro

AT HOME — on your laptop

- Understand precision: 32-bit (precise), 16-bit (faster, tiny loss), 8-bit (fastest, needs care).
- Write down the FP16 export commands for both models so you can paste them in the lab.

IN THE LAB — on the Orin

- Build both engines with FP16; re-benchmark against the baseline; identify which stage eats the frame budget.

CARRY TO THE LAB A clear understanding of precision and the ready-to-paste export commands.

DAY 12 — INT8 Quantization + Calibration

Biggest speedup; calibrate on the Orin • MOSTLY LAB

Goal. Try INT8 and weigh the speed gain against accuracy loss.

THE GIST — what you're really doing

INT8 calibration **must run on the Orin** (the mapping is hardware-specific), but the **calibration set** — a few hundred representative images — you assemble at home. So the prep is home work; the calibration run is lab work.

READINGS

- INT8 export & calibration — Ultralytics

AT HOME — on your laptop

- Assemble a representative calibration set from your captures (a few hundred frames).
- Understand what calibration does, and write the `data.yaml` that points to those images.

IN THE LAB — on the Orin

- Export INT8 engine(s) **on the Orin** with calibration; compare INT8 vs FP16 for speed and accuracy; decide what to keep.

CARRY TO THE LAB A prepared calibration image set and the data.yaml, ready to calibrate.

DAY 13 — Resolution & Pipeline-Shape Tuning

Get inside the budget (mixed) • HOME + LAB

Goal. Find the smallest resolution and leanest shape that still works.

THE GIST — what you're really doing

You can explore the **accuracy** side of resolution at home — does YOLO still find your objects at 640 instead of 1280? — but the **speed** numbers need the Orin. So narrow down candidate resolutions at home, then benchmark them for speed in the lab.

READINGS

- Re-skim the Jetson benchmarking guide for the resolution-vs-speed trade-off.

AT HOME — on your laptop

- On your laptop, run YOLO at several input resolutions and check which still detects what you need.
- Confirm your pipeline segments only the boxes YOLO found, never the whole frame.

IN THE LAB — on the Orin

- Benchmark each candidate resolution for speed on the Orin; pick the one that fits the 100 ms budget.

CARRY TO THE LAB A shortlist of candidate resolutions with accuracy notes.

DAY 14 — Wrap as a ROS 2 Node

Make it deployable — mostly home work • MOSTLY HOME

Goal. Turn the pipeline into a ROS 2 node, tested at home.

THE GIST — what you're really doing

This is one of the best days to do at home, because **ROS 2 runs on a normal Ubuntu laptop**. You can install it, learn publish/subscribe, and write and test your entire node using a **fake image publisher** (or a recorded bag file) that feeds it sample images. On the Orin you simply point the same node at the real camera topic. The deployment glue is built and debugged before you ever touch the device.

READINGS

- ROS 2 publisher/subscriber (Python)
- ROS 2 for Beginners

AT HOME — on your laptop

- Install ROS 2 on your laptop (easiest on Ubuntu; on Windows/Mac use a VM or Docker — or do this part on the Orin if your laptop isn't Ubuntu).
- Do the publisher/subscriber tutorial until you can write both.
- Write your node: subscribe to an image topic, run the pipeline in the callback, publish the result.
- Test it with a fake publisher that sends sample images, or by playing a recorded bag — no camera needed.

IN THE LAB — on the Orin

- Run the same node against the real camera topic on the Orin.

CARRY TO THE LAB A working, tested ROS 2 node (Git) that only needs to be pointed at the real camera.

DAY 15 — Integration + Live Test

Prove it in the real stack (lab) • MOSTLY LAB

Goal. Run live inside the full SCANNER-01 system.

THE GIST — what you're really doing

Live integration is a device task — it has to run alongside the real camera and the other projects' nodes, competing for resources. At home you **prepare**: confirm the interfaces and calibration version with Projects B and C.

READINGS

- Coordinate with Project B (fusion) and Project C (data store) on interfaces and calibration version.

AT HOME — on your laptop

- Review the topic names and message types you'll connect to; note open questions for B and C.

IN THE LAB — on the Orin

- Run your node in the live stack; do a walk-through; watch `tegrastats` for throttling and frame drops; fix the worst issues. Write the Week 3 update.

CARRY TO THE LAB Interface notes and questions, so integration time isn't spent discovering basics.

WEEK 4

3D, evaluation & hand-off

DAY 16

Lift 2D Masks onto 3D Points

The projection math — home-testable • MOSTLY HOME

Goal. Write and verify the code that puts 2D labels onto 3D points.

THE GIST — what you're really doing

The projection math — take a 3D point, work out which camera pixel it lands on, copy that pixel's label onto the point — is **pure code you can write and verify at home** using made-up (synthetic) point data and a known calibration, so you can check the answer by hand. On the Orin you plug in Project B's real calibration and the real LiDAR. The hard part (getting the geometry right) is best done calmly at home.

READINGS

- Point cloud basics — MathWorks
- Project B's camera–LiDAR calibration — confirm the version (lab).

AT HOME — on your laptop

- Learn how a point cloud is stored: arrays of (x, y, z).
- Write the projection: transform a point by the extrinsics, project with the intrinsics, look up the pixel's label.
- Test it with synthetic points and a known calibration so you can verify correctness by hand.

IN THE LAB — on the Orin

- Plug in Project B's real calibration and the LiDAR; build the labelled cloud; view it in RViz/Foxglove.

CARRY TO THE LAB Tested projection code (Git) that needs only the real calibration numbers.

DAY 17

Multi-Camera Conflicts + Project C

Resolve disagreements; persist (mixed) • HOME + LAB

Goal. Decide conflicts between cameras and save labels with recordings.

THE GIST — what you're really doing

The conflict-resolution logic (what to do when two cameras label the same point differently) is **home-codeable** and testable with made-up data. The persistence side needs Project C's interface, so that part is lab work.

READINGS

- Project C's plugin / recording interface.

AT HOME — on your laptop

- Write the conflict rule (vote, highest-confidence, or weight by confidence) and test it with synthetic multi-camera labels.

IN THE LAB — on the Orin

- Integrate with Project C's data store and persist labels alongside recordings.

CARRY TO THE LAB Tested conflict-resolution logic (Git).

DAY

Final Evaluation

18 — *Numbers for quality and speed (mostly home)* • MOSTLY HOME

Goal. Score the pipeline's quality; record its real-time speed.

THE GIST — what you're really doing

Running the accuracy evaluation (mIoU on your held-out set) is **laptop work**. Only the final inference-speed number — the real-time claim — has to be measured on the Orin. So you do most of the evaluation at home and grab the one speed figure in the lab.

READINGS

- A refresher on how mIoU is computed from a confusion matrix.

AT HOME — on your laptop

- Run mIoU evaluation on the locked held-out set, per class; write an honest limitations section.

IN THE LAB — on the Orin

- Measure the final inference rate on the Orin — the real-time number for your report.

CARRY TO THE LAB A near-complete evaluation report; only the on-Orin speed figure is added in the lab.

DAY 19 — Documentation & Reproducibility

Make it runnable by a stranger (home) • MOSTLY HOME

Goal. Write the README and tidy the repository.

THE GIST — what you're really doing

Writing the README and cleaning up the repo is **entirely home work**. The aim is that someone could go from a fresh Orin to a running demo using only your README.

READINGS

- Your own repo — read it as a stranger and note where they'd get stuck.

AT HOME — on your laptop

- Write a step-by-step README from fresh Orin to running demo.
- Document the engine-build commands, calibration steps, launch commands, and where files live; tidy the repo.

IN THE LAB — on the Orin

- If possible, verify the steps on the Orin in a clean shell to catch missing details.

CARRY TO THE LAB A complete README and a tidy repo.

DAY 20 — Presentation & Hand-Off

Land it well (mostly home) • MOSTLY HOME

Goal. Build the talk and hand the project over cleanly.

THE GIST — what you're really doing

Building the 15-minute walkthrough and the “if I had another month” notes is **home work**. The one device-bound task is capturing a demo video on the Orin as a backup in case the live demo misbehaves.

READINGS

- Re-read your weekly updates to reconstruct the project's arc.

AT HOME — on your laptop

- Build the 15-minute walkthrough: what was built, what failed, what's next. Write the “next month” list.

IN THE LAB — on the Orin

- Record a demo video on the Orin as a backup.

CARRY TO THE LAB A finished presentation and a clean hand-off.

5. The short version

Develop the logic at home, carry it to the Orin for the device-specific and speed-specific steps, and move it with Git.

- **Big home wins:** Days 4 (glue), 8 (labelling), 9 (robustness), 14 (ROS 2 node), 16 (projection math), 18–20 (evaluation, docs, talk). These are most of the project.
- **Truly needs the Orin:** building engines (Days 2, 3, 11, 12), the camera and live stack (Days 7, 15), and final speed numbers (Days 5, 18).
- **The bridge:** write code at home → `git push` → `git pull` on the Orin → run. Keep engines and datasets out of Git.
- **If stuck on the Orin's setup,** keep building the home-doable days in parallel — you do not have to wait for the device to make progress.

A note on the SAM stand-in

Several home days suggest using SAM or MobileSAM on your laptop as a stand-in for NanoSAM, since NanoSAM only runs on the Orin. This is a sound way to develop the pipeline logic remotely — but confirm with your lead that it's worth the setup time, and remember the final system uses NanoSAM on the device. The glue code stays the same; only the segmenter object is swapped.